`calendar` — General calendar-related functions

Source code: Lib/calendar.py

This module allows you to output calendars like the Unix **cal** program, and provides additional useful functions related to the calendar. By default, these calendars have Monday as the first day of the week, and Sunday as the last (the European convention). Use `setfirstweekday()` to set the first day of the week to Sunday (6) or to any other weekday. Parameters that specify dates are given as integers. For related functionality, see also the `datetime` and `time` modules.

The functions and classes defined in this module use an idealized calendar, the current Gregorian calendar extended indefinitely in both directions. This matches the definition of the "proleptic Gregorian" calendar in Dershowitz and Reingold's book "Calendrical Calculations", where it's the base calendar for all computations. Zero and negative years are interpreted as prescribed by the ISO 8601 standard. Year 0 is 1 BC, year -1 is 2 BC, and so on.

class calendar.**Calendar**(*firstweekday=0*)

Creates a `Calendar` object. *firstweekday* is an integer specifying the first day of the week. `MONDAY` is `0` (the default), `SUNDAY` is `6`.

A `Calendar` object provides several methods that can be used for preparing the calendar data for formatting. This class doesn't do any formatting itself. This is the job of subclasses.

`Calendar` instances have the following methods and attributes:

firstweekday

The first weekday as an integer (0–6).

This property can also be set and read using `setfirstweekday()` and `getfirstweekday()` respectively.

getfirstweekday()

Return an `int` for the current first weekday (0–6).

Identical to reading the `firstweekday` property.

setfirstweekday(*firstweekday*)

Set the first weekday to *firstweekday*, passed as an `int` (0–6).

Identical to setting the `firstweekday` property.

iterweekdays()

Return an iterator for the weekday numbers that will be used for one week. The first value from the iterator will be the same as the value of the `firstweekday` property.

itermonthdates(*year, month*)

Return an iterator for the month *month* (1–12) in the year *year*. This iterator will return all days (as `datetime.date` objects) for the month and all days before the start of the month or after the end of the month that are required to get a complete week.

itermonthdays(*year, month*)

Return an iterator for the month *month* in the year *year* similar to `itermonthdates()`, but not restricted by the `datetime.date` range. Days returned will simply be day of the month numbers. For the days outside of the specified month, the day number is `0`.

itermonthdays2(*year, month*)

Return an iterator for the month *month* in the year *year* similar to `itermonthdates()`, but not restricted by the `datetime.date` range. Days returned will be tuples consisting of a day of the month number and a weekday number.

itermonthdays3(*year, month*)

Return an iterator for the month *month* in the year *year* similar to `itermonthdates()`, but not restricted by the `datetime.date` range. Days returned will be tuples consisting of a year, a month and a day of the month numbers.

> *Added in version 3.7.*

itermonthdays4(*year, month*)

Return an iterator for the month *month* in the year *year* similar to `itermonthdates()`, but not restricted by the `datetime.date` range. Days returned will be tuples consisting of a year, a month, a day of the month, and a day of the week numbers.

> *Added in version 3.7.*

monthdatescalendar(*year, month*)

Return a list of the weeks in the month *month* of the *year* as full weeks. Weeks are lists of seven `datetime.date` objects.

monthdays2calendar(*year, month*)

Return a list of the weeks in the month *month* of the *year* as full weeks. Weeks are lists of seven tuples of day numbers and weekday numbers.

monthdayscalendar(*year, month*)

Return a list of the weeks in the month *month* of the *year* as full weeks. Weeks are lists of seven day numbers.

yeardatescalendar(*year, width=3*)

Return the data for the specified year ready for formatting. The return value is a list of month rows. Each month row contains up to *width* months (defaulting to 3). Each month contains between 4 and 6 weeks

and each week contains 1–7 days. Days are `datetime.date` objects.

yeardays2calendar(*year, width=3*)

Return the data for the specified year ready for formatting (similar to `yeardatescalendar()`). Entries in the week lists are tuples of day numbers and weekday numbers. Day numbers outside this month are zero.

yeardayscalendar(*year, width=3*)

Return the data for the specified year ready for formatting (similar to `yeardatescalendar()`). Entries in the week lists are day numbers. Day numbers outside this month are zero.

class calendar.**TextCalendar**(*firstweekday=0*)

This class can be used to generate plain text calendars.

`TextCalendar` instances have the following methods:

formatday(*theday, weekday, width*)

Return a string representing a single day formatted with the given *width*. If *theday* is `0`, return a string of spaces of the specified width, representing an empty day. The *weekday* parameter is unused.

formatweek(*theweek, w=0*)

Return a single week in a string with no newline. If *w* is provided, it specifies the width of the date columns, which are centered. Depends on the first weekday as specified in the constructor or set by the `setfirstweekday()` method.

formatweekday(*weekday, width*)

Return a string representing the name of a single weekday formatted to the specified *width*. The *weekday* parameter is an integer representing the day of the week, where `0` is Monday and `6` is Sunday.

formatweekheader(*width*)

Return a string containing the header row of weekday names, formatted with the given *width* for each column. The names depend on the locale settings and are padded to the specified width.

formatmonth(*theyear, themonth, w=0, l=0*)

Return a month's calendar in a multi-line string. If *w* is provided, it specifies the width of the date columns, which are centered. If *l* is given, it specifies the number of lines that each week will use. Depends on the first weekday as specified in the constructor or set by the `setfirstweekday()` method.

formatmonthname(*theyear, themonth, width=0, withyear=True*)

Return a string representing the month's name centered within the specified *width*. If *withyear* is `True`, include the year in the output. The *theyear* and *themonth* parameters specify the year and month for the name to be formatted respectively.

prmonth(*theyear, themonth, w=0, l=0*)

Print a month's calendar as returned by `formatmonth()`.

formatyear(*theyear, w=2, l=1, c=6, m=3*)

Return a *m*-column calendar for an entire year as a multi-line string. Optional parameters *w*, *l*, and *c* are for date column width, lines per week, and number of spaces between month columns, respectively. Depends on the first weekday as specified in the constructor or set by the `setfirstweekday()` method. The earliest year for which a calendar can be generated is platform-dependent.

pryear(*theyear, w=2, l=1, c=6, m=3*)

Print the calendar for an entire year as returned by `formatyear()`.

class calendar.**HTMLCalendar**(*firstweekday=0*)

This class can be used to generate HTML calendars.

`HTMLCalendar` instances have the following methods:

formatmonth(*theyear, themonth, withyear=True*)

Return a month's calendar as an HTML table. If *withyear* is true the year will be included in the header, otherwise just the month name will be used.

formatyear(*theyear, width=3*)

Return a year's calendar as an HTML table. *width* (defaulting to 3) specifies the number of months per row.

formatyearpage(*theyear, width=3, css='calendar.css', encoding=None*)

Return a year's calendar as a complete HTML page. *width* (defaulting to 3) specifies the number of months per row. *css* is the name for the cascading style sheet to be used. `None` can be passed if no style sheet should be used. *encoding* specifies the encoding to be used for the output (defaulting to the system default encoding).

formatmonthname(*theyear, themonth, withyear=True*)

Return a month name as an HTML table row. If *withyear* is true the year will be included in the row, otherwise just the month name will be used.

`HTMLCalendar` has the following attributes you can override to customize the CSS classes used by the calendar:

cssclasses

A list of CSS classes used for each weekday. The default class list is:

```
cssclasses = ["mon", "tue", "wed", "thu", "fri", "sat", "sun"]
```

more styles can be added for each day:

```
cssclasses = ["mon text-bold", "tue", "wed", "thu", "fri", "sat", "sun red"]
```

Note that the length of this list must be seven items.

cssclass_noday

The CSS class for a weekday occurring in the previous or coming month.

> *Added in version 3.7.*

cssclasses_weekday_head

A list of CSS classes used for weekday names in the header row. The default is the same as `cssclasses`.

> *Added in version 3.7.*

cssclass_month_head

The month's head CSS class (used by `formatmonthname()`). The default value is `"month"`.

> *Added in version 3.7.*

cssclass_month

The CSS class for the whole month's table (used by `formatmonth()`). The default value is `"month"`.

> *Added in version 3.7.*

cssclass_year

The CSS class for the whole year's table of tables (used by `formatyear()`). The default value is `"year"`.

> *Added in version 3.7.*

cssclass_year_head

The CSS class for the table head for the whole year (used by `formatyear()`). The default value is `"year"`.

> *Added in version 3.7.*

Note that although the naming for the above described class attributes is singular (e.g. `cssclass_month` `cssclass_noday`), one can replace the single CSS class with a space separated list of CSS classes, for example:

```
"text-bold text-red"
```

Here is an example how `HTMLCalendar` can be customized:

```
class CustomHTMLCal(calendar.HTMLCalendar):
    cssclasses = [style + " text-nowrap" for style in
                  calendar.HTMLCalendar.cssclasses]
    cssclass_month_head = "text-center month-head"
    cssclass_month = "text-center month"
    cssclass_year = "text-italic lead"
```

class calendar.**LocaleTextCalendar**(*firstweekday=0, locale=None*)

This subclass of `TextCalendar` can be passed a locale name in the constructor and will return month and weekday names in the specified locale.

class calendar.**LocaleHTMLCalendar**(*firstweekday=0, locale=None*)

This subclass of `HTMLCalendar` can be passed a locale name in the constructor and will return month and weekday names in the specified locale.

Note: The constructor, `formatweekday()` and `formatmonthname()` methods of these two classes temporarily change the `LC_TIME` locale to the given *locale*. Because the current locale is a process-wide setting, they are not thread-safe.

For simple text calendars this module provides the following functions.

calendar.**setfirstweekday**(*weekday*)

Sets the weekday (`0` is Monday, `6` is Sunday) to start each week. The values `MONDAY`, `TUESDAY`, `WEDNESDAY`, `THURSDAY`, `FRIDAY`, `SATURDAY`, and `SUNDAY` are provided for convenience. For example, to set the first weekday to Sunday:

```
import calendar
calendar.setfirstweekday(calendar.SUNDAY)
```

calendar.**firstweekday**()

Returns the current setting for the weekday to start each week.

calendar.**isleap**(*year*)

Returns `True` if *year* is a leap year, otherwise `False`.

calendar.**leapdays**(*y1, y2*)

Returns the number of leap years in the range from *y1* to *y2* (exclusive), where *y1* and *y2* are years.

This function works for ranges spanning a century change.

calendar.**weekday**(*year, month, day*)

Returns the day of the week (`0` is Monday) for *year* (`1970`–...), *month* (`1`–`12`), *day* (`1`–`31`).

calendar.**weekheader**(*n*)

Return a header containing abbreviated weekday names. *n* specifies the width in characters for one weekday.

calendar.**monthrange**(*year, month*)

Returns weekday of first day of the month and number of days in month, for the specified *year* and *month*.

calendar.**monthcalendar**(*year, month*)

Returns a matrix representing a month's calendar. Each row represents a week; days outside of the month are represented by zeros. Each week begins with Monday unless set by `setfirstweekday()`.

calendar.**prmonth**(*theyear, themonth, w=0, l=0*)

Prints a month's calendar as returned by `month()`.

calendar.**month**(*theyear, themonth, w=0, l=0*)

Returns a month's calendar in a multi-line string using the `formatmonth()` of the `TextCalendar` class.

calendar.**prcal**(*year, w=0, l=0, c=6, m=3*)

Prints the calendar for an entire year as returned by `calendar()`.

calendar.**calendar**(*year, w=2, l=1, c=6, m=3*)

Returns a 3-column calendar for an entire year as a multi-line string using the `formatyear()` of the `TextCalendar` class.

calendar.**timegm**(*tuple*)

An unrelated but handy function that takes a time tuple such as returned by the `gmtime()` function in the `time` module, and returns the corresponding Unix timestamp value, assuming an epoch of 1970, and the POSIX encoding. In fact, `time.gmtime()` and `timegm()` are each other's inverse.

The `calendar` module exports the following data attributes:

calendar.**day_name**

A sequence that represents the days of the week in the current locale, where Monday is day number 0.

```
>>> import calendar
>>> list(calendar.day_name)
['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
```

calendar.**day_abbr**

A sequence that represents the abbreviated days of the week in the current locale, where Mon is day number 0.

```
>>> import calendar
>>> list(calendar.day_abbr)
['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']
```

calendar.**MONDAY**
calendar.**TUESDAY**
calendar.**WEDNESDAY**
calendar.**THURSDAY**
calendar.**FRIDAY**
calendar.**SATURDAY**
calendar.**SUNDAY**

Aliases for the days of the week, where `MONDAY` is `0` and `SUNDAY` is `6`.

> *Added in version 3.12.*

class calendar.**Day**

Enumeration defining days of the week as integer constants. The members of this enumeration are exported to the module scope as `MONDAY` through `SUNDAY`.

> *Added in version 3.12.*

calendar.**month_name**

A sequence that represents the months of the year in the current locale. This follows normal convention of January being month number 1, so it has a length of 13 and `month_name[0]` is the empty string.

```
>>> import calendar
>>> list(calendar.month_name)
['', 'January', 'February', 'March', 'April', 'May', 'June', 'July', 'August', 'September'
```

calendar.**month_abbr**

A sequence that represents the abbreviated months of the year in the current locale. This follows normal convention of January being month number 1, so it has a length of 13 and `month_abbr[0]` is the empty string.

```
>>> import calendar
>>> list(calendar.month_abbr)
['', 'Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
```

calendar.**JANUARY**
calendar.**FEBRUARY**
calendar.**MARCH**
calendar.**APRIL**
calendar.**MAY**
calendar.**JUNE**
calendar.**JULY**
calendar.**AUGUST**
calendar.**SEPTEMBER**
calendar.**OCTOBER**
calendar.**NOVEMBER**
calendar.**DECEMBER**

Aliases for the months of the year, where `JANUARY` is `1` and `DECEMBER` is `12`.

> *Added in version 3.12.*

class calendar.**Month**

Enumeration defining months of the year as integer constants. The members of this enumeration are exported to the module scope as `JANUARY` through `DECEMBER`.

> *Added in version 3.12.*

The `calendar` module defines the following exceptions:

exception calendar.**IllegalMonthError**(*month*)

A subclass of `ValueError` and `IndexError`, raised when the given month number is outside of the range 1-12 (inclusive).

> *Changed in version 3.12:* `IllegalMonthError` is now also a subclass of `ValueError`. New code should avoid catching `IndexError`.

month

The invalid month number.

exception calendar.**IllegalWeekdayError**(*weekday*)

A subclass of `ValueError`, raised when the given weekday number is outside of the range 0-6 (inclusive).

weekday

The invalid weekday number.

See also:

Module `datetime`

Object-oriented interface to dates and times with similar functionality to the `time` module.

Module `time`

Low-level time related functions.

Command-line usage

Added in version 2.5.

The `calendar` module can be executed as a script from the command line to interactively print a calendar.

```
python -m calendar [-h] [-L LOCALE] [-e ENCODING] [-t {text,html}]
                   [-w WIDTH] [-l LINES] [-s SPACING] [-m MONTHS] [-c CSS]
                   [-f FIRST_WEEKDAY] [year] [month]
```

For example, to print a calendar for the year 2000:

```
$ python -m calendar 2000
                                  2000

      January                   February                  March
Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su
                1  2          1  2  3  4  5  6            1  2  3  4  5
 3  4  5  6  7  8  9       7  8  9 10 11 12 13       6  7  8  9 10 11 12
10 11 12 13 14 15 16      14 15 16 17 18 19 20      13 14 15 16 17 18 19
17 18 19 20 21 22 23      21 22 23 24 25 26 27      20 21 22 23 24 25 26
24 25 26 27 28 29 30      28 29                     27 28 29 30 31
31

       April                     May                      June
Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su
                1  2       1  2  3  4  5  6  7                1  2  3  4
 3  4  5  6  7  8  9       8  9 10 11 12 13 14       5  6  7  8  9 10 11
10 11 12 13 14 15 16      15 16 17 18 19 20 21      12 13 14 15 16 17 18
17 18 19 20 21 22 23      22 23 24 25 26 27 28      19 20 21 22 23 24 25
24 25 26 27 28 29 30      29 30 31                  26 27 28 29 30

        July                    August                 September
Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su
                1  2          1  2  3  4  5  6                   1  2  3
 3  4  5  6  7  8  9       7  8  9 10 11 12 13       4  5  6  7  8  9 10
10 11 12 13 14 15 16      14 15 16 17 18 19 20      11 12 13 14 15 16 17
17 18 19 20 21 22 23      21 22 23 24 25 26 27      18 19 20 21 22 23 24
24 25 26 27 28 29 30      28 29 30 31               25 26 27 28 29 30
31

      October                   November                  December
Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su
                   1             1  2  3  4  5                   1  2  3
 2  3  4  5  6  7  8       6  7  8  9 10 11 12       4  5  6  7  8  9 10
 9 10 11 12 13 14 15      13 14 15 16 17 18 19      11 12 13 14 15 16 17
16 17 18 19 20 21 22      20 21 22 23 24 25 26      18 19 20 21 22 23 24
23 24 25 26 27 28 29      27 28 29 30               25 26 27 28 29 30 31
30 31
```

The following options are accepted:

--help, -h

Show the help message and exit.

--locale `LOCALE`**, -L** `LOCALE`

The locale to use for month and weekday names. Defaults to English.

--encoding `ENCODING`**, -e** `ENCODING`

The encoding to use for output. `--encoding` is required if `--locale` is set.

--type `{text,html}`**, -t** `{text,html}`

Print the calendar to the terminal as text, or as an HTML document.

--first-weekday `FIRST_WEEKDAY`**, -f** `FIRST_WEEKDAY`

The weekday to start each week. Must be a number between 0 (Monday) and 6 (Sunday). Defaults to 0.

Added in version 3.13.

year

The year to print the calendar for. Defaults to the current year.

month

The month of the specified `year` to print the calendar for. Must be a number between 1 and 12, and may only be used in text mode. Defaults to printing a calendar for the full year.

Text-mode options:

--width `WIDTH`**, -w** `WIDTH`

The width of the date column in terminal columns. The date is printed centred in the column. Any value lower than 2 is ignored. Defaults to 2.

--lines `LINES`**, -l** `LINES`

The number of lines for each week in terminal rows. The date is printed top-aligned. Any value lower than 1 is ignored. Defaults to 1.

--spacing `SPACING`**, -s** `SPACING`

The space between months in columns. Any value lower than 2 is ignored. Defaults to 6.

--months `MONTHS`**, -m** `MONTHS`

The number of months printed per row. Defaults to 3.

> *Changed in version 3.14:* By default, today's date is highlighted in color and can be controlled using environment variables.

HTML-mode options:

`--css` CSS**, `-c`** CSS
: The path of a CSS stylesheet to use for the calendar. This must either be relative to the generated HTML, or an absolute HTTP or `file:///` URL.